

Received 8 July 2025, accepted 11 August 2025, date of publication 14 August 2025, date of current version 20 August 2025.

Digital Object Identifier 10.1109/ACCESS.2025.3598858

RESEARCH ARTICLE

CAXF-LCCDE: An Enhanced Feature Extraction and Ensemble Learning Model for XSS Detection

MAN HUA¹, ZHIYU LUAN¹, YANLING LI², AND QIJIAN LIU¹

¹School of Computer Science, Civil Aviation Flight University of China, Guanghan, Sichuan 618307, China

²School of Science, Civil Aviation Flight University of China, Guanghan, Sichuan 618307, China

Corresponding author: Zhiyu Luan (17861539985@163.com)

This work was supported in part by Sichuan Science and Technology Department under Grant 2023YFG0171.

ABSTRACT Threats to businesses, organizations, and individuals have become more complicated and varied due to the constant evolution of cyberattack strategies brought about by the quick development of Internet technologies. Cross-Site Scripting (XSS) remains one of the most pervasive and dangerous threats among the many that target online application security. This emphasizes the importance of strong detection mechanisms to protect user information and preserve system integrity. Combining machine learning with intrusion detection systems successfully addresses this problem and enhances XSS detection capabilities. This work provides a new feature extraction technique designed specifically for XSS, called Comprehensive Analysis of XSS Features (CAXF). We combine it with Leader Class and Confidence Decision Ensemble (LCCDE), an ensemble machine learning approach. This combination efficiently identifies various XSS attack types, such as reflected, stored, and DOM-based variations. Extensive trials were carried out on well-annotated datasets with typical samples of these XSS categories. The findings demonstrate that the suggested CAXF-LCCDE framework significantly outperforms conventional resilience and detection accuracy models. Furthermore, the results highlight how crucial tailored feature engineering and algorithm selection are for cybersecurity applications. This work provides important information for creating stronger and more dependable XSS attack defenses.

INDEX TERMS Feature extraction, machine learning algorithms, XSS attacks, intrusion detection, cybersecurity.

I. INTRODUCTION

A. RESEARCH BACKGROUND AND MOTIVATIONS

With the widespread adoption of web applications across domains such as e-commerce, social media, and online collaboration, their cross-platform compatibility and convenience have driven the rapid development of Internet services [1]. However, the large volume of sensitive user information embedded within these systems has also made them prime targets for cyberattacks, leading to risks of data breaches and privacy violations that threaten both system security and individual rights [2], [3], [4].

Against this backdrop, the development of efficient intrusion detection systems (IDS) has become increasingly important. Intelligent IDS solutions can identify and respond

to potential attacks in real time, thereby enhancing overall network resilience and protecting critical infrastructure. In particular, as attack strategies continue to evolve, research into adaptive and automated detection methods is essential to address the growing complexity of cybersecurity threats.

Among these threats, Cross-Site Scripting (XSS) remains one of the most prevalent and dangerous web vulnerabilities. XSS exploits occur primarily due to improper handling of user inputs by web applications, allowing attackers to inject malicious scripts that are subsequently executed in the user's browser [5]. According to the Open Web Application Security Project (OWASP), XSS ranked among the top ten most critical web security risks in 2013, 2017, and 2021, appearing as the 3rd, 7th, and again the 3rd most severe vulnerability, respectively [6]. This consistent presence underscores the persistent and severe impact of XSS on

The associate editor coordinating the review of this manuscript and approving it for publication was Shaohua Wan.

user privacy and application integrity, especially in sensitive contexts such as financial services, social platforms, and enterprise systems.

To mitigate XSS attacks, researchers have proposed a range of countermeasures, including input validation, output encoding, security policies, and browser-based protection mechanisms. In recent years, advances in machine learning and deep learning have driven significant progress in XSS detection. By leveraging feature modeling and behavioral analysis, artificial intelligence techniques can effectively identify potential malicious scripts, enabling more automated and accurate detection capabilities [7], [8].

B. CURRENT RESEARCH STATUS OF XSS ATTACKS

Given the high dimensionality and inherent complexity of XSS data, feature extraction during the preprocessing phase is crucial. The choice of preprocessing methods significantly impacts detection performance and should be carefully considered when designing XSS detection approaches [9]. Current feature extraction methods for XSS include the Universal Sentence Encoder (USE), an effective tool that turns textual information into high-dimensional vectors, reflecting the underlying meanings and relationships, and models such as Word2vec. While these models perform well in detecting a broad range of intrusion attacks, they fall short in achieving finer granularity when specifically applied to XSS attack variants.

Moreover, XSS attacks are commonly classified into stored, reflected, and DOM-based types, each of which is exploited by attackers through distinct mechanisms. However, during the training of ML models, significant variations in predictive performance are observed across different XSS attack types [10], making it challenging to achieve optimal detection results.

Additionally, in the current academic research landscape, datasets specifically tailored for XSS attacks are scarce, and even fewer provide detailed categorizations of XSS attack types. In XSS detection research, the granular classification of XSS datasets is critical [5]. The lack of finely segmented XSS datasets not only hinders the model's ability to fully capture the characteristics of different attack types but also impacts the formulation of defense strategies, the evaluation of model performance, the analysis of attack patterns, and the interpretability of models. This limitation poses a significant obstacle to ongoing academic research and development in this field [11].

C. CONTRIBUTIONS

We have methodically approached and improved each issue in order to address the previously listed ones. The following is a summary of this paper's contributions:

(1) By constructing a novel feature extraction method for XSS payloads, termed CAXF, we have achieved a comprehensive feature representation of cross-site attack payloads. Our proposed method aims to capture fine-grained semantic

meanings and more diverse feature representations, including HTML tags, JavaScript events, URL counts, special character counts, occurrences of specific keywords, and TF-IDF vectorized representations of text. This improves feature extraction and boosts model performance. We carried out a series of in-depth experiments with machine learning models to assess the effectiveness of our method. Comparative results with traditional feature extraction methods such as USE and Word2Vec demonstrate that our method achieves higher accuracy, precision, recall, and F1 score in detecting XSS attacks. Additionally, we assessed the contribution of each feature to the model by visualizing the importance of each feature.

(2) LCCDE, originally proposed in [12] for intrusion detection in the Internet of Vehicles, was adapted and integrated with CAXF for XSS attack detection, yielding promising predictive results. LCCDE is a novel ensemble framework for intrusion detection systems (IDS) designed to enhance detection performance by selecting the most effective machine learning model among XGBoost, LightGBM, and CatBoost for each specific type of attack. The framework then utilizes class leader models and their prediction confidence levels to make accurate decisions for detecting various types of cyberattacks.

We further adapted LCCDE to better integrate with CAXF and enhance its visualization capabilities. Specifically, we refactored the LCCDE functions to include probability output support, enabling the prediction probabilities of samples to be output. Additionally, we averaged the probability results from the three base models and calculated their feature importance to provide graphical references for the multi-dimensional features in CAXF. A unified interface was implemented to support methods such as fit, predict, and score, aligning with the scikit-learn API standards.

(3) We employed the XSS deep learning dataset made publicly available by Syed Saqlain Hussain Shah on the Kaggle platform. Based on this dataset, we precisely categorized the data into stored, reflected, and DOM-based XSS types according to the attack location, execution process of the attack chain, and the execution method of malicious code. This refined classification expanded the label categories, providing a more granular dataset for research.

II. LITERATURE REVIEW

Significant progress has been made in the detection of XSS attacks in recent years. Researchers have spent a lot of time investigating ways to detect XSS assaults, and they are gradually moving toward machine learning techniques that use sophisticated models and algorithms to learn from massive amounts of data and identify distinctive patterns of XSS attacks [6].

The authors of [13] proposed algoXSSF, a hybrid detection framework that applies classical machine learning algorithms to identify and analyze both Cross-Site Scripting and Cross-Site Request Forgery (XSRF) attacks. The approach involved extracting a rich set of lexical and structural features from HTTP request payloads, enabling the model to capture

patterns associated with malicious behavior. Various classifiers, including Decision Trees, Random Forests, and Support Vector Machines, were evaluated for their effectiveness in detecting cross-site vulnerabilities. Notably, the system achieved high detection accuracy while maintaining low false positive rates, demonstrating strong generalization across different attack scenarios. The study highlights the importance of integrating multiple classifiers and well-engineered feature sets for robust web attack mitigation.

In another study, the authors of [14] introduced an innovative approach that leveraged modular deep neural networks to minimize false positive rates in XSS detection systems. Word2Vec was used to extract semantically related terms from a large-scale text corpus. The model, specifically tailored for XSS attack identification and prevention, utilized 50 features selected through the Pearson correlation coefficient.

By fusing deep learning and swarm intelligence, Kumar et al. [15] looked into phishing website identification. Their method uses the binary bat algorithm to differentiate between malicious and legitimate URLs and the Adam optimizer to speed up training. Thirty features were extracted for classification, including attributes like URL length, IP address presence, user ID, iframe tags, port numbers, redirection, and mouse hover behavior.

Feature extraction plays a vital role in cybersecurity, contributing to more accurate, efficient, and adaptable threat detection systems [16]. For example, a study [17] proposed an autoencoder-based feature extraction framework to classify Android malware applications. Similarly, [18] introduced a method that transforms multiple feature sources, including DEX files and API call graphs, into RGB images for malware detection using image classification techniques. In [19], a pre-trained Roberta model was used to extract text features for detecting spam on social media platforms, enhancing a stacked BLSTM network's performance. Additionally, transformer-based models such as BERT, DistilBERT, ALBERT, and Electra were employed to capture textual features. Another study [20] adopted deep contextual ELMo embeddings for feature extraction. For comparative analysis, traditional embedding methods like Word2Vec, GloVe, and FastText were also incorporated into experimental evaluations.

Although many related works have achieved high performance in XSS detection tasks, there remains significant room for improvement. By employing more advanced feature extraction methods and integrated ML algorithm strategies, we can enhance existing predictive metrics. To the best of our knowledge, the proposed CAXF-LCCDE framework is the first comprehensive approach applied to stored, reflected, and DOM-based XSS attacks, delivering superior results in terms of Accuracy, Precision, Recall, and F1-score. The goal of this integrated method is to deliver a powerful and practical solution for real-time detection of cross-site attacks in web applications, adding value to existing research and advancing the field of web security.

III. RESEARCH METHOD

The suggested cross-site attack detection model's comprehensive framework is shown in this section. Our novel strategy consists of three primary parts: (1) the **feature extraction procedure**, which extracts important features from the dataset using the built CAXF. (2) **splitting the dataset** into training and testing sets using a 70:30 split and applying the Synthetic Minority Over-sampling Technique (SMOTE) to reduce **class imbalance**; (3) using the learning model LCCDE for prediction, which completes the integration process with CAXF. A thorough visual overview of the complete architecture is given in Figure 1.

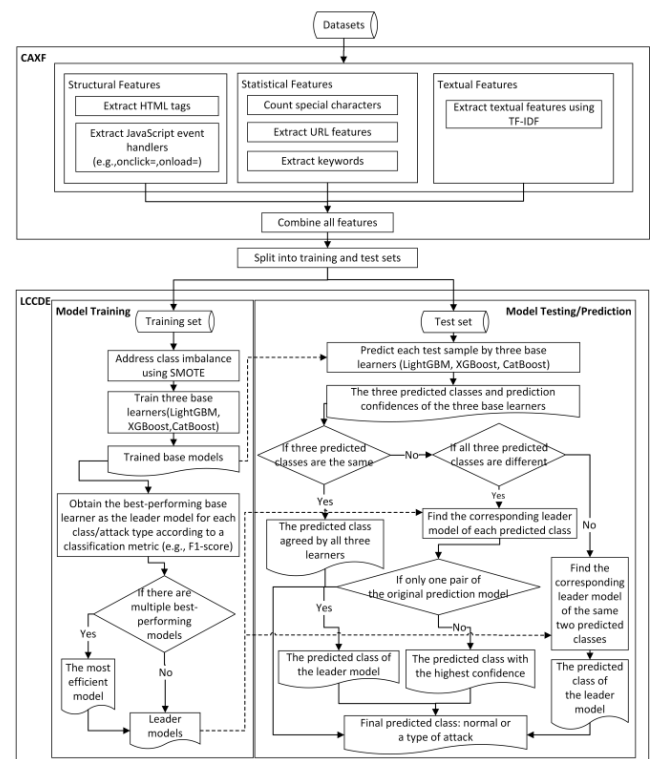


FIGURE 1. The overall architecture of the proposed CAXF-LCCDE framework.

The model consists of **two main components**: feature extraction and ensemble learning, which are seamlessly integrated. **First**, CAXF extracts three categories of features (comprising six sub-features in total). After the extraction process is completed, the features are combined and organized to provide critical input for the subsequent LCCDE process. LCCDE, on the other hand, generates the **final prediction class** by integrating three base learners, using the leader model and confidence scores as optimization criteria. The combination of CAXF and LCCDE optimizes intrusion detection for XSS attacks, significantly enhancing overall performance.

A. CAXF

Feature extraction is essential for detecting cross-site attacks. It involves converting raw input data (e.g., web content)

Algorithm 1 Comprehensive Analysis of XSS Features (CAXF) Feature Extraction**Data:** Input: XSS data set containing payload results**Result:** Output: Extracted features and combined Embedding

- 1: Tags = HTML parser(XSS)
- 2: Extract(Tags)
- 3: Events = regex(XSS)
- 4: Returns the extracted event handler
- 5: URLs = regex(XSS)
- 6: Count occurrences of Special Characters
- 7: Returns the count in the dictionary assigned to SC
- 8: Define a list of XSS-related keywords
- 9: The number of occurrences is calculated and assigned to KW
- 10: TF-IDF Vectorizer
- 11: TF = TF-IDF.fit_transform(XSS)
- 12: Extract and merge feature items
- 13: Store in Embedding Column

into structured formats that machine learning algorithms can process efficiently [16].

CAXF consists of six sub-features that capture structural, behavioral, and semantic patterns relevant to XSS detection:

(1) HTML Tag Structure: Identifies and analyzes the structure of embedded markup elements to detect irregular nesting patterns commonly associated with malicious behaviors [21];

(2) JavaScript Event Triggers: Detects attributes used for scripting event handling (typically prefixed with “on”), which may indicate attacks initiated through user interaction;

(3) URL Invocation Frequency: Measures the occurrence of complete hyperlink references within the input, reflecting potential reliance on external malicious resources [22];

(4) Special Character Distribution: Examines the frequency of syntax-related special characters typically used in web scripting, which may contribute to the construction of obfuscated or suspicious code [23];

(5) Malicious Keyword Frequency: Calculates the frequency of context-specific lexical indicators commonly associated with script-based attacks, enabling the construction of a high-dimensional semantic representation that reflects potential intent for code execution or data exfiltration;

(6) Character-Level TF-IDF Vectorization: Utilizes a sliding window-based TF-IDF technique to extract character-level n-grams, effectively capturing partial identifiers, encoded function patterns, and structural fragments, thereby enhancing the model’s robustness against obfuscated or minimally modified payloads [24].

Together, these six components construct a multi-dimensional representation that enhances the model’s ability to identify complex and evasive XSS patterns.

Based on the six sub-features, the CAXF extraction process begins by parsing HTML tags from the input samples, which helps the model identify common HTML structures typically associated with XSS attacks. Subsequently, JavaScript event

handlers are extracted using regular expressions, which aids in distinguishing normal text from potentially malicious code. Taking the extraction of structural features as an example, HTML tag features can be represented as follows:

$$\text{Tags}(S) = \{t_1, t_2, \dots, t_k\} \quad (1)$$

$$f_{\text{tags}}(S) = [\text{freq}(t_1), \text{freq}(t_2), \dots, \text{freq}(t_k)] \quad (2)$$

Here, $\text{Tags}(S)$ represents the foundation of feature extraction, identifying the HTML tags present. $f_{\text{tags}}(S)$ denotes the generation of quantitative feature vectors based on these tags, which are then utilized by machine learning models.

The presence of URLs may indicate potential XSS attack attempts, as certain attacks could be executed by redirecting to malicious sites. Therefore, extracting URL features is also a critical factor. Subsequently, given the characteristics of XSS attack payloads, identifying special characters, extracting keywords, and calculating their frequency in the payload are further steps. Additionally, we employ **TF-IDF (Term Frequency-Inverse Document Frequency)** as a significant feature reference. TF-IDF is a statistical method used to measure the importance of words in text, widely applied in information retrieval and text analysis. It reflects the relative importance of words in a document by combining term frequency and inverse document frequency, making it particularly suitable for extracting information from large text corpora [25]. Using TF-IDF, samples are transformed into sparse matrix representations, enabling **character-level text analysis** and capturing character sequences through word boundaries, effectively reducing noise and enhancing model performance. Finally, the extracted feature arrays are combined to form a comprehensive feature matrix.

$$F(S) = [f_{\text{tags}}(S), f_{\text{eve}}(S), f_{\text{url}}(S), f_{\text{sc}}(S), f_{\text{kw}}(S), f_{\text{tf}}(S)] \quad (3)$$

$$\text{Embedding}_i = F(S_i), \forall S_i \in \text{df}[\text{'Sent'}] \quad (4)$$

All extracted features are concatenated to form a final comprehensive feature vector. The dimensionality of the features generated by TF-IDF is p , while the total dimensionality of features such as HTML tags and event handlers is q . Thus, the final feature vector has a dimensionality of $F(S) \in \mathbb{R}^{p+q}$. This comprehensive feature vector is then stored in the column named Embedding.

B. SMOTE

Before training, we employed SMOTE (Synthetic Minority Over-sampling Technique) to address the class imbalance problem. By generating synthetic samples (rather than simply duplicating existing ones), SMOTE introduces greater diversity, effectively reducing the risk of overfitting and enhancing the model’s generalization capability and classification performance [26]. Here, SMOTE was used to oversample the data points of reflected XSS attack samples and DOM-based XSS attack samples to the average level. Experimental results demonstrate that the inclusion of SMOTE led to improvements in Precision, Recall, and F1-Score.

C. LCCDE

After completing the preparatory work, the LCCDE framework is employed for training and prediction. This ensemble model is constructed using three advanced gradient-boosting ML methods: XGBoost, LightGBM, and CatBoost. The process involves the following steps:

(1) Training the Base Learners: To create the base learners, the training set is used to train the three base machine learning models, XGBoost, LightGBM, and CatBoost.

(2) Assessing the Base Learners: Cross-validation and the F1-score, which was chosen for its comprehensiveness and efficiency in managing unbalanced datasets, are used to assess each model's performance across several classes (normal or attack type).

(3) Choosing the Leader Model for Every Class: The model with the highest F1-score is selected as the leader for every class. The fastest and most effective model is chosen as the ultimate leader when several models obtain the same maximum F1-score.

Following training, predictions are made using each class's leader models. The model with the highest prediction confidence is selected for the final selection when there are several leader models available for various classes. The generated ensemble model performs optimally in identifying different kinds of XSS attacks by utilizing CAXF-LCCDE.

Building upon the LCCDE framework proposed in [12], we made adaptive improvements to facilitate better integration with CAXF and enhance its visualization capabilities. Specifically, the corresponding LCCDE functions were refactored, and a probability output support method was added to enable the prediction probabilities of samples to be output. Additionally, the probability results from the three base models were averaged and integrated. The feature importance of the three base models was also calculated to provide graphical references for the multi-dimensional features in CAXF. A unified interface was implemented to support methods such as fit, predict, and score, ensuring compliance with the scikit-learn API standards.

IV. MODEL PERFORMANCE EVALUATION

A. DESCRIPTION OF DATASETS

In this work, we added to the XSS deep learning dataset that Syed Saqlain Hussain Shah made freely accessible on the Kaggle repository. In order to provide a strong basis for our cross-site attack detection tests and assessments, we further classified the data according to XSS attack categories, building on the original dataset, which comprises 13,685 instances of both benign samples and XSS attacks. We can evaluate the performance of our model in a variety of attack scenarios thanks to the dataset, which contains attack vectors for stored XSS, reflected XSS, and DOM-based XSS, in addition to a sizable number of benign vectors. With the use of this dataset, we were able to thoroughly assess the capabilities and effectiveness of our XSS attack detection techniques, producing enlightening findings.

Algorithm 2 Leader Class and Confidence Decision Ensemble(LCCDE)

Data: Input: Dec (Embedding Column after CAXF)
 $M = \{M_1, M_2, M_3\}$: the base ML model list, including $M_1 = \text{LightGBM}$, $M_2 = \text{XGBoost}$, $M_3 = \text{CatBoost}$,
 $c = 1, 2, \dots, n$: the class list for n different classes.
Result: Output: L_{test} : the prediction classes for all test samples in D_{test} .

For each class(normal or a type of attack):
 1: Use M_1, M_2, M_3 to train
 $M_1' \leftarrow \text{Training}(M_1, D_{\text{ec}})$
 $M_2' \leftarrow \text{Training}(M_2, D_{\text{ec}})$
 $M_3' \leftarrow \text{Training}(M_3, D_{\text{ec}})$
 2: Determine which model performs the best in
 each class (for example, the one with the greatest F1-score).
End For
For each test sample:
 3: Use M_1', M_2', M_3' to predict
 save the predicted class L_{i1}, L_{i2}, L_{i3} & confidence p_{i1}, p_{i2}, p_{i3}
 4: **If** $L_{i1} == L_{i2} == L_{i3}$, the predictions from all three models are identical
 Use this predicted class as the final predicted class
 5: **If** $L_{i1} != L_{i2} != L_{i3}$ all three models produce distinct predictions
 For each prediction model:
 If the predicted class matches the prediction of its corresponding leader model
 $L_list_i \leftarrow L_list_i \cup \{L_{i,j}\}$ Save the predicted class
 $p_list_i \leftarrow p_list_i \cup \{p_{i,j}\}$ Save the confidence
 End For
 If only one model has a consistent prediction with its leader model
 Use the leader model's predicted class as the final output
 Else
 Select the class with the highest confidence score as the final prediction
 6: **If** two models agree on the same class while the third differs
 $n \leftarrow \text{mode}(L_{i1}, L_{i2}, L_{i3})$ Find the predicted class with the majority vote
 $L_i \leftarrow \text{Prediction}(M_n, x_i)$ Use the leader model's prediction corresponding to class n as the final predicted label
 7: $L_{\text{test}} \leftarrow L_{\text{test}} \cup \{L_i\}$ Append the determined class L_i to the L_{test} list for all test samples.
End For

TABLE 1. Characteristics of XSS types.

Attack Type	Malicious Script Storage	Attack Process	Target
Stored XSS	Stored on the server	Code is stored long-term; triggered when users access the page	Server and Client
Reflected XSS	Not stored	Transmitted via URL or request parameters	Client
DOM-Based XSS	Not stored	Dynamically generated by client-side scripts	Client (Front-end Code)

Persistent XSS, sometimes referred to as stored XSS, happens when an attacker leaves malicious scripts on the target server. The harmful script is automatically run when other users visit the website that contains it [27]. Reflected XSS is a non-persistent attack where the malicious script is not

stored on the server but is instead delivered through request parameters. The attacker lures the victim into visiting a specially crafted link, causing the script to execute immediately in the user's browser [5]. DOM-based XSS attacks directly manipulate client-side scripts (e.g., JavaScript). The attacker modifies the DOM structure of the page, causing the page to execute malicious scripts [28], [29].

B. EXPERIMENT SETUP

The XSS attack detection model's performance and efficacy were evaluated using a wide range of evaluation indicators. These metrics give a complete picture of the model's performance by assessing its accuracy, precision, recall, and F1-score. To help identify any biases or imbalances in the model's functioning, a confusion matrix was also used to display the distribution of true positives, true negatives, false positives, and false negatives. Furthermore, an enhanced confusion matrix was applied with normalization to display numerical proportions, addressing the issue where the original confusion matrix might exaggerate the performance of the majority class in imbalanced datasets. Execution time, including CPU runtime, wall-clock time, and per-sample runtime, was used to assess the model's efficiency. Feature importance plots were generated to illustrate the influence of the six categories of features in CAXF on the model's decision-making. Finally, learning curves were plotted to show the trends of training error and validation error as a function of training set size.

C. MODEL DEPLOYMENT

To evaluate the deployability of the proposed CAXF-LCCDE intrusion detection framework in real-world edge environments, a series of resource consumption and computational efficiency analyses were conducted. The implementation of the CAXF-LCCDE model was carried out in Python using standard machine learning libraries, including Scikit-learn, XGBoost, LightGBM, and CatBoost. All experiments were executed on a Dell Precision 3630 workstation equipped with an Intel Core i7-8700 processor and 16 GB of RAM.

In terms of memory usage, the combined model—which includes three base learners and the TF-IDF vocabulary—occupies less than 50 MB when serialized. This makes it compatible with embedded platforms featuring 512 MB to 2 GB of RAM, such as IoT gateways or edge devices with constrained resources.

D. DEPLOYMENT CONSIDERATIONS

Modern Web Application Firewalls (WAFs) play a vital role in protecting web applications by filtering and monitoring HTTP traffic. The CAXF-LCCDE framework can be deployed as a plug-in detection module within WAF architectures by intercepting and analyzing HTTP request payloads before they reach the backend application logic.

Since the model operates solely on extracted payload features and does not require complete browser behavior

or session history context, it supports stateless inspection, making it well-suited for inline analysis. In addition, the modular design of CAXF allows for seamless integration with WAF rule enhancement mechanisms. For instance, alerts generated by CAXF-LCCDE can be used to dynamically update WAF rule sets, thereby enabling adaptive defense against evolving attack vectors.

Given its low latency and efficient memory usage, the model can be embedded directly into reverse proxies or edge computing gateways without introducing significant performance overhead.

E. EXPERIMENTAL RESULTS

In terms of feature extraction, due to the unique characteristics of XSS attack types, conventional feature extraction methods have proven ineffective in distinguishing between XSS and normal traffic, as well as differentiating among various categories of XSS attacks. The hybrid model combining USE and Word2Vec, as mentioned in [30], has achieved significant results in the XSS domain, substantially improving the efficiency of extracting meaningful features. In this paper, we compare the experimental results of USE-Word2Vec with the proposed CAXF, as shown in the table below, through a comprehensive comparison of four evaluation metrics across three base ML models.

TABLE 2. Performance comparison of models after feature extraction using USE-Word2Vec.

Classifier	Accuracy	Precision	Recall	Average F1-score
LightGBM	0.9519	0.9540	0.9515	0.9517
XGBoost	0.9532	0.9554	0.9532	0.9534
CatBoost	0.9532	0.9559	0.9532	0.9534
LCCDE	0.9525	0.9550	0.9525	0.9527

TABLE 3. Performance comparison of models after feature extraction using CAXF.

Classifier	Accuracy	Precision	Recall	Average F1-score
LightGBM	0.9938	0.9932	0.9939	0.9930
XGBoost	0.9949	0.9951	0.9943	0.9945
CatBoost	0.9925	0.9927	0.9920	0.9923
LCCDE	0.9964	0.9965	0.9962	0.9963

Currently, most research focuses on detecting normal traffic and XSS attack payloads, while further classification within the current domain remains underexplored. Taking USE-Word2Vec as an example, the F1-scores for different types of samples were calculated and compared with the classification results of CAXF.

TABLE 4. Comparison of F1-scores for different XSS attack types after feature extraction using USE-Word2Vec.

Classifier	F1 (%) of Class 1: Normal	F1 (%) of Class 2: Stored XSS	F1 (%) of Class 3: Reflected XSS	F1 (%) of Class 4: DOM-Based XSS
LightGBM	0.9897	0.9050	0.8809	0.9822
XGBoost	0.9894	0.9066	0.8872	0.9845
CatBoost	0.9878	0.9103	0.8885	0.9830
LCCDE	0.9894	0.9062	0.8850	0.9830

TABLE 5. Comparison of F1-scores for different XSS attack types after feature extraction using CAXF.

Classifier	F1 (%) of Class 1: Normal	F1 (%) of Class 2: Stored XSS	F1 (%) of Class 3: Reflected XSS	F1 (%) of Class 4: DOM-Based XSS
LightGBM	0.9972	0.9945	0.9915	0.9934
XGBoost	0.9976	0.9942	0.9937	0.9941
CatBoost	0.9970	0.9907	0.9894	0.9926
LCCDE	0.9978	0.9956	0.9960	0.9948

It can be observed that certain feature extraction methods fail to provide suitable parameters for specific types of XSS attacks, resulting in the model's inability to correctly distinguish the target categories. In contrast, the F1-scores generated using CAXF feature extraction are very close to 1, demonstrating its strong representational capability for XSS attack types.

Additionally, we note that the F1-scores for each of the four models assessed according to F1-scores for each attack type vary amongst basic ML models for identifying distinct attack types. For instance, LightGBM performs better than XGBoost in detecting stored XSS assaults, but XGBoost achieves the highest F1-score among the three base learners in recognizing normal samples, reflected XSS attacks, and DOM-based XSS attacks. The suggested LCCDE ensemble model has the highest F1-score in each category, as the table illustrates.

Additionally, the performance of these deep learning architectures exhibits significant differences when combined with different feature extraction techniques. The proposed combination of CAXF and LCCDE demonstrates notably superior results.

When examining the temporal analysis results, it is evident that the combination of CAXF and LCCDE demonstrates superior runtime efficiency compared to other feature extraction methods. The representation storage mechanism transformed through CAXF feature extraction is more concise and efficient, saving significant time. The time efficiency of CAXF can be attributed to the high parallelization among its

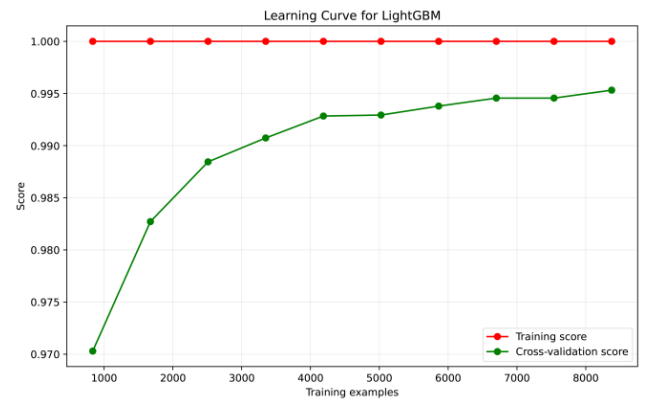
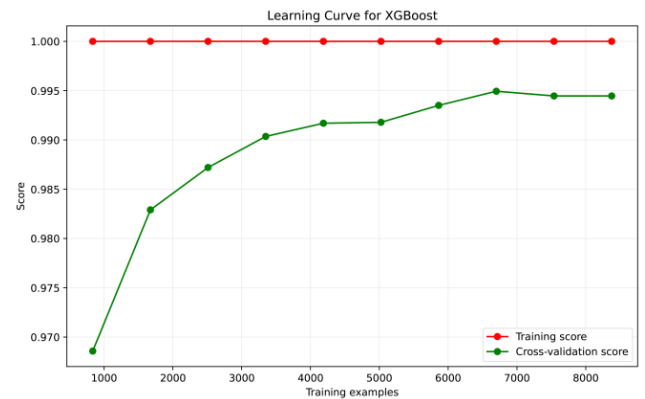
TABLE 6. Temporal analysis results after feature extraction using USE-Word2Vec.

Classifier	CPU Times(sec)	Wall time(sec)	Time/sample (ms)
LightGBM	51.1	17	1.623
XGBoost	170	26.6	2.540
CatBoost	1122	219	20.91
LCCDE	186	129	12.31

TABLE 7. Temporal analysis results after feature extraction using CAXF.

Classifier	CPU Times(sec)	Wall time(sec)	Time/sample (ms)
LightGBM	3.05	2.54	0.242
XGBoost	9.02	2.37	0.226
CatBoost	86	30.7	2.932
LCCDE	96	80	7.640

multiple sub-feature extraction processes and the advantage of not requiring pre-trained models.

**FIGURE 2.** Learning curve for XSS attack detection using CAXF(LightGBM).**FIGURE 3.** Learning curve for XSS attack detection using CAXF(XGBoost).

As the number of training samples increases, the learning curves show the models' performance trends on both the training and cross-validation sets. All four models show

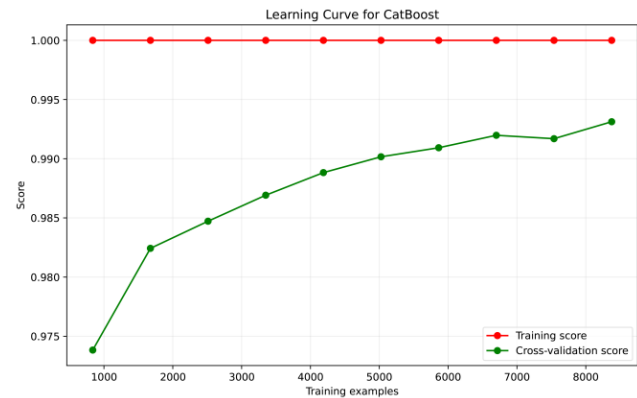


FIGURE 4. Learning curve for XSS attack detection using CAXF(CatBoost).

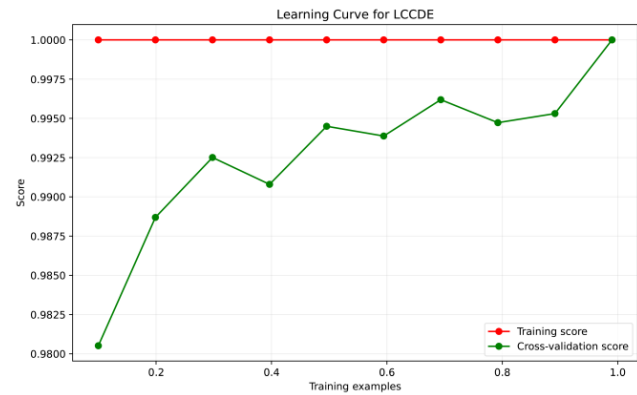


FIGURE 5. Learning curve for XSS attack detection using CAXF(LCCDE).

good training and cross-validation scores under CAXF feature extraction, as can be seen from the four learning curve plots. This suggests that the features recovered by CAXF are very useful for the XSS attack detection task. This is particularly evident in the compatibility with LCCDE. The cross-validation score of LCCDE almost coincides with its training score, demonstrating its superior generalization capability. This fully illustrates that the overall model framework of CAXF-LCCDE can maximize its advantages.

The normalized confusion matrix is derived by normalizing the standard confusion matrix, where the values represent relative ratios rather than absolute sample counts. By observation, under CAXF feature extraction, the diagonal elements of the normalized confusion matrix are close to 1, indicating that the majority of samples are correctly classified with an extremely low misclassification rate. This demonstrates that the model exhibits strong classification performance under the CAXF feature extraction approach. Furthermore, the classification performance shows minimal variation, with almost consistent accuracy across all categories. This suggests that the features extracted by CAXF exhibit strong adaptability across different ML models, are not reliant on any specific model, and are suitable for various learning

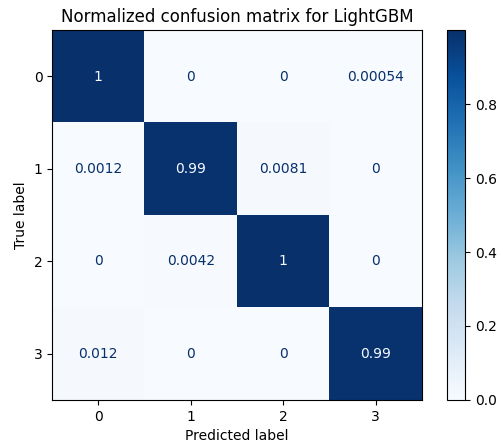


FIGURE 6. Normalized confusion matrix for XSS attack detection using CAXF(LightGBM).

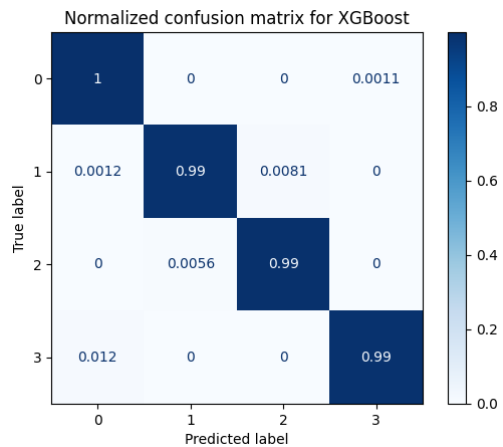


FIGURE 7. Normalized confusion matrix for XSS attack detection using CAXF(XGBoost).

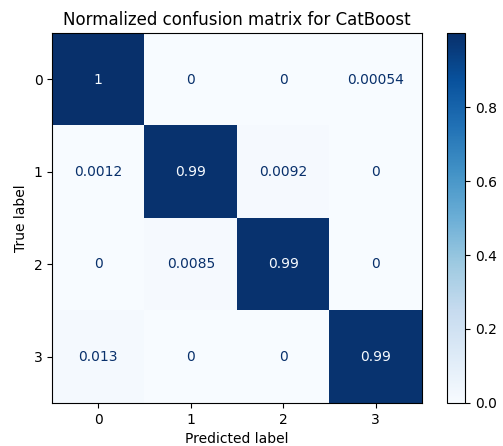


FIGURE 8. Normalized confusion matrix for XSS attack detection using CAXF(CatBoost).

algorithms, demonstrating excellent generalization capability for identifying XSS attack types.

Under the CAXF feature extraction method, the feature importance graphs obtained from four learning models

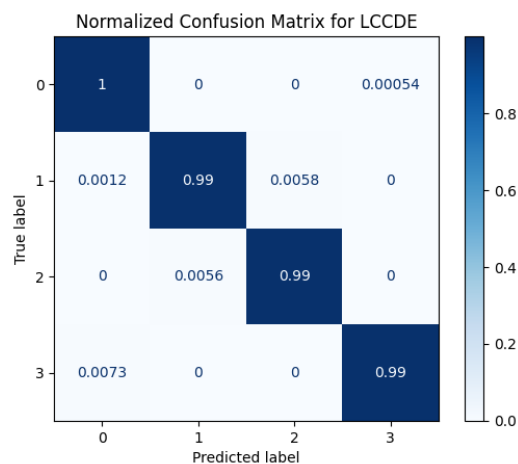


FIGURE 9. Normalized confusion matrix for XSS attack detection using CAXF(LCCDE).

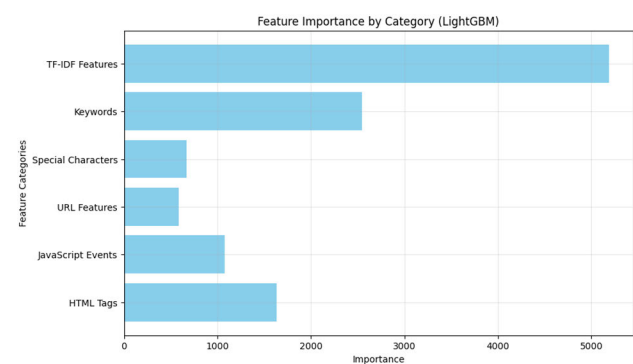


FIGURE 10. Feature importance plot for XSS attack detection using CAXF(LightGBM).

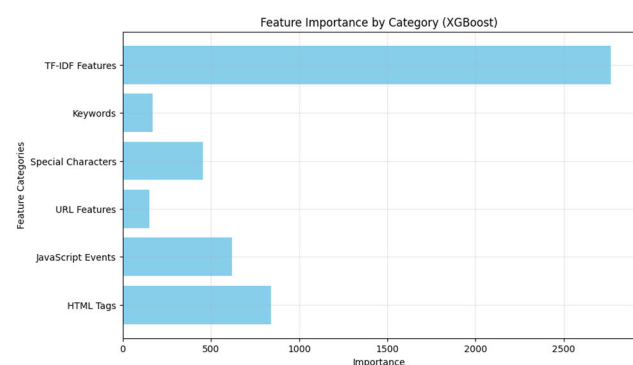


FIGURE 11. Feature importance plot for XSS attack detection using CAXF(XGBoost).

indicate that among structural features, statistical features, and textual features, the proportion of importance assigned to TF-IDF-based text feature extraction is the highest. This suggests that all four models exhibit a strong reliance on textual features, demonstrating their accuracy and high adaptability in XSS prediction. Additionally, since different types of XSS attacks depend on distinct informational features,

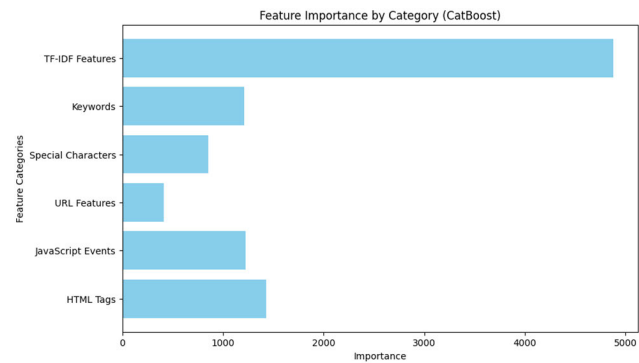


FIGURE 12. Feature importance plot for XSS attack detection using CAXF(CatBoost).

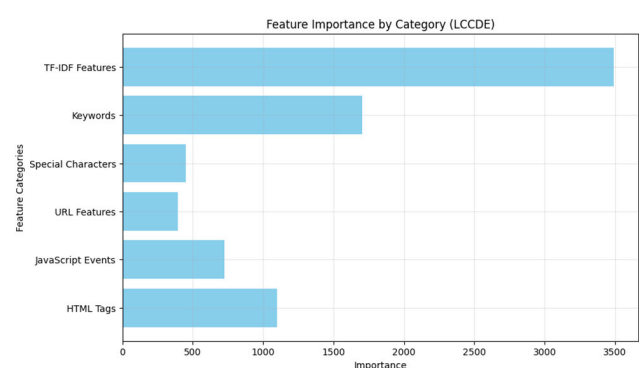


FIGURE 13. Feature importance plot for XSS attack detection using CAXF(LCCDE).

other sub-feature extraction methods exhibit unique advantages across different models. Overall, multimodal feature fusion enhances detection coverage, whereas relying on a single feature type may lead to false positives or false negatives. For instance, using only TF-IDF may struggle to detect code obfuscation attacks, while relying solely on structural features may fail to identify semantic-level attack patterns. The integrated feature extraction approach, which combines both semantic and structural information, enables models to detect both attack vector variations and semantic modifications. This enhances adaptability and generalization to emerging XSS variants, effectively reducing false positive and false negative rates while improving model robustness.

TABLE 8. Overall model performance comparison.

Detection Method	Accuracy	Precision	Recall	Average F1-score
CNN + Attention	0.9785	0.9754	0.9732	0.9743
BERT + Dense Layer	0.9837	0.9819	0.9788	0.9803
XSS-GUARD	0.9382	0.9311	0.9254	0.9282
CAXF-LCCDE	0.9964	0.9965	0.9962	0.9963

After separately validating the advantages of the CAXF feature extraction strategy and the LCCDE ensemble decision mechanism in XSS detection, the integration of both into a unified detection framework—CAXF-LCCDE—demonstrates even more pronounced and comprehensive performance improvements.

To further assess its practical effectiveness, we conduct a comparative analysis against three widely acknowledged and representative XSS detection paradigms: (1) a CNN + Attention model that captures local structural dependencies and emphasizes salient input features, representing the dominant direction in deep learning-based intrusion detection; (2) a BERT + Dense Layer architecture built upon transformer-based pretrained language models, which has shown strong capabilities in semantic understanding and context-aware classification; and (3) XSS-GUARD, a hybrid approach combining syntactic analysis with manually crafted detection rules, commonly adopted in rule-based Web Application Firewalls (WAFs) and industrial intrusion prevention systems.

As shown in Table 8, while all three baseline methods exhibit strong performance in detecting standard XSS patterns, the proposed CAXF-LCCDE framework achieves the highest scores across all four evaluation metrics—Accuracy, Precision, Recall, and Average F1-score. This highlights its superior generalization capability and robustness, particularly in distinguishing between fine-grained XSS variants. Unlike BERT-based models, CAXF-LCCDE maintains excellent accuracy without incurring heavy computational overhead, and its structural feature integration allows for more resilient detection of obfuscated or mutated payloads that may bypass purely semantic or rule-based methods.

V. CONCLUSION

Effectively reducing the impact of XSS assaults, stopping illegal access or data breaches, and safeguarding the integrity of web applications all depend on how quickly they are discovered. Even though there are many feature extraction techniques and machine learning algorithms for intrusion detection, it is still very difficult to effectively detect XSS attacks and provide accurate performance metrics.

Moreover, given the diversity of current XSS attack methods, different XSS types exhibit substantial differences in attack vectors, execution environments, and detection difficulty. A single detection approach is insufficient to comprehensively cover all XSS variants, which is one of the core challenges in XSS detection. Therefore, accurately distinguishing XSS types is essential for improving detection accuracy and coverage, enhancing attack traceability, and increasing the adaptive capabilities of defense systems.

The effectiveness of XSS detection largely relies on efficient feature extraction and advanced algorithmic techniques, which are crucial for rapidly identifying and mitigating XSS threats. Experiments were conducted using multiple metrics, and the importance of each sub-feature in CAXF was analyzed across different ML models. The results demonstrate

that the proposed CAXF-LCCDE significantly outperforms traditional methods, not only improving the accuracy of XSS attack detection but also highlighting the potential of specific feature extraction and ML algorithms in cybersecurity applications. This offers important insights for creating stronger and more effective security measures to defend against XSS attacks.

Future work should focus on improving detection techniques, optimizing real-time deployment, and expanding defense mechanisms against emerging XSS variants. Emphasizing the study of XSS attack categories is crucial, as a comprehensive understanding of XSS types can not only enhance detection performance but also play a pivotal role in advancing web security. By tailoring defense mechanisms to specific attack types, refining machine learning models, and integrating adaptive security strategies, researchers and practitioners can build more resilient and intelligent XSS detection systems. Ultimately, this will reduce the risk of vulnerability exploitation and strengthen the security of modern web applications.

REFERENCES

- [1] G. Jošt and V. Taneski, "State-of-the-art cross-platform mobile application development frameworks: A comparative study of market and developer trends," *Informatics*, vol. 12, no. 2, p. 45, Apr. 2025.
- [2] M. A. I. Mallick and R. Nath, "Navigating the cyber security landscape: A comprehensive review of cyber-attacks, emerging trends, and recent developments," *World Sci. News*, vol. 190, no. 1, pp. 1–69, 2024.
- [3] Ö. Aslan, S. S. Aktuğ, M. Ozkan-Okay, A. A. Yilmaz, and E. Akin, "A comprehensive review of cyber security vulnerabilities, threats, attacks, and solutions," *Electronics*, vol. 12, no. 6, p. 1333, Mar. 2023.
- [4] S. Aswathy and A. K. Tyagi, "Privacy breaches through cyber vulnerabilities: Critical issues, open challenges, and possible countermeasures for the future," in *Security and Privacy-Preserving Techniques in Wireless Robotics*. Boca Raton, FL, USA: CRC Press, 2022, pp. 163–210.
- [5] A. Hannousse, S. Yahiouche, and M. C. Nait-Hamoud, "Twenty-two years since revealing cross-site scripting attacks: A systematic mapping and a comprehensive survey," *Comput. Sci. Rev.*, vol. 52, May 2024, Art. no. 100634.
- [6] I. K. Thajeel, K. Samsudin, S. J. Hashim, and F. Hashim, "Machine and deep learning-based XSS detection approaches: A systematic literature review," *J. King Saud Univ. Comput. Inf. Sci.*, vol. 35, no. 7, Jul. 2023, Art. no. 101628.
- [7] A. Sohail, B. Ayisha, I. Hameed, M. Mohsin Zafar, H. Alquhayz, and A. Khan, "Deep neural networks based meta-learning for network intrusion detection," 2023, *arXiv:2302.09394*.
- [8] N.-C. Pistol, B. Alexandrescu, and R.-C. Constantinescu, "Integrated and automated intrusion detection system: A comprehensive review of current challenges and methods, future trends and prospects for emerging electronics engineering with AI integration," in *Proc. Int. Symp. Electron. Telecommun. (ISETC)*, Nov. 2024, pp. 1–4.
- [9] Q. Qin, Y. Li, Y. Mi, J. Shen, K. Wu, and Z. Wang, "Detecting XSS with random forest and multi-channel feature extraction," *Comput., Mater. Continua*, vol. 80, no. 1, pp. 843–874, 2024.
- [10] K. H. Hamzah, M. Z. Osman, T. Anthony, M. A. Ismail, Z. Abdullah, and A. Alanda, "Comparative analysis of machine learning algorithms for cross-site scripting (XSS) attack detection," *JOIV Int. J. Informat. Visualizat.*, vol. 8, nos. 3–2, pp. 1678–1685, Nov. 2024.
- [11] G.-H. Luu, M.-K. Duong, T.-P. Pham-Ngo, T.-S. Ngo, D.-T. Nguyen, X.-H. Nguyen, and K.-H. Le, "XSShield: A novel dataset and lightweight hybrid deep learning model for XSS attack detection," *Results Eng.*, vol. 24, Dec. 2024, Art. no. 103363.
- [12] L. Yang, A. Shami, G. Stevens, and S. D. Rusett, "LCCDE: A decision-based ensemble framework for intrusion detection in the Internet of Vehicles," in *Proc. IEEE Global Commun. Conf.*, Dec. 2022, pp. 3545–3550.

- [13] N. Kshetri, D. Kumar, J. Hutson, N. Kaur, and O. F. Osama, "AlgoXSSF: Detection and analysis of cross-site request forgery (XSRF) and cross-site scripting (XSS) attacks via machine learning algorithms," in *Proc. 12th Int. Symp. Digit. Forensics Secur. (ISDFS)*, Apr. 2024, pp. 1–8.
- [14] A. A. Alqarni, N. Alsharif, N. A. Khan, L. Georgieva, E. Pardede, and M. Y. Alzahrani, "MNN-XSS: Modular neural network-based approach for XSS attack detection," *Computers, Materials & Continua*, vol. 70, no. 2, pp. 4075–4085, 2022.
- [15] P. Pavan Kumar, T. Jaya, and V. Rajendran, "Si-BBA—A novel phishing website detection based on swarm intelligence with deep learning," *Materials Today, Proc.*, vol. 80, no. 3, pp. 3129–3139, 2023.
- [16] M. H. Chandana, G. Dorasanamma, S. Kiran, and A. A. Kumar, "A review on feature extraction techniques using machine learning," *Macaw Int. J. Adv. Res. Comput. Sci. Eng.*, vol. 10, no. 1, pp. 57–63, 2024.
- [17] H. Bakır and R. Bakır, "DroidEncoder: Malware detection using auto-encoder based feature extractor and machine learning algorithms," *Comput. Electr. Eng.*, vol. 110, Sep. 2023, Art. no. 108804.
- [18] Z. Wang, Q. Yu, and S. Yuan, "Android malware detection based on RGB images and multi-feature fusion," 2024, *arXiv:2408.16555*.
- [19] R. Ghanem, H. Erbay, and K. Bakour, "Contents-based spam detection on social networks using RoBERTa embedding and stacked BLSTM," *Social Netw. Comput. Sci.*, vol. 4, no. 4, p. 380, May 2023.
- [20] R. Ghanem and H. Erbay, "Spam detection on social networks using deep contextualized word representation," *Multimedia Tools Appl.*, vol. 82, no. 3, pp. 3697–3712, Jan. 2023.
- [21] Y. Nakagawa and M. Mimura, "XSS attack detection by attention mechanism based on script tags in URLs," in *Proc. Int. Conf. Inf. Secur. Pract. Exper.* (Lecture Notes in Computer Science). Singapore: Springer, 2023, pp. 562–578.
- [22] Z. Hu, J. Zhang, and H. Yang, "XSS attack detection based on multisource semantic feature fusion," *Electronics*, vol. 14, no. 6, p. 1174, Mar. 2025.
- [23] X. Li, T. Wang, W. Zhang, X. Niu, T. Zhang, T. Zhao, Y. Wang, and Y. Wang, "An LSTM based cross-site scripting attack detection scheme for cloud computing environments," *J. Cloud Comput.*, vol. 12, no. 1, p. 118, Aug. 2023.
- [24] K. Tamamura, S. Sakai, K. Watarai, S. Okada, and T. Mitsunaga, "Detection of XSS attacks with one class SVM using TF-IDF and devising a vectorized vocabulary," in *Proc. IEEE Int. Conf. Comput. (ICOCO)*, Oct. 2023, pp. 35–40.
- [25] R. N. Rath and A. Mustafi, "The importance of term weighting in semantic understanding of text: A review of techniques," *Multimedia Tools Appl.*, vol. 82, no. 7, pp. 9761–9783, Mar. 2023.
- [26] H. Sharma and A. Gosain, "Oversampling methods to handle the class imbalance problem: A review," in *Proc. Int. Conf. Soft Comput. Eng. Appl. (ICICCE)*, vol. 1. Singapore: Springer, Jan. 2023, pp. 96–110.
- [27] I. Khazal and M. Hussain, "Server side method to detect and prevent stored XSS attack," *Iraqi J. Electr. Electron. Eng.*, vol. 17, no. 2, pp. 58–65, Dec. 2021.
- [28] J. Hickling, "What is DOM XSS and why should you care?" *Comput. Fraud Secur.*, vol. 2021, no. 4, pp. 6–10, Jan. 2021.
- [29] W. Melicher, C. Fung, L. Bauer, and L. Jia, "Towards a lightweight, hybrid approach for detecting DOM XSS vulnerabilities with machine learning," in *Proc. Web Conf.*, Apr. 2021, pp. 2684–2695.
- [30] R. Bakır and H. Bakır, "Swift detection of XSS attacks: Enhancing XSS attack detection by leveraging hybrid semantic embeddings and AI techniques," *Arabian J. Sci. Eng.*, vol. 50, no. 2, pp. 1–17, Jan. 2025.



MAN HUA received the master's degree in cryptography from Southwest Jiaotong University, Chengdu, China, in 2004. He is currently with the School of Computer Science, Civil Aviation Flight University of China, Deyang. His research interests include cryptography, cybersecurity, computer vision, and image processing.



ZHIYU LUAN is currently pursuing the master's degree in network and information security with the School of Computer Science, Civil Aviation Flight University of China. His research interests include the intersection of cybersecurity vulnerabilities and feature extraction.



YANLING LI received the master's degree in applied mathematics from China University of Geosciences (Wuhan), Wuhan, China, in 2008. She is currently with the School of Science, Civil Aviation Flight University of China, Deyang. Her research interests include applied mathematics, artificial neural networks, and computer vision.



QIJIAN LIU received the master's degree in software engineering from the University of Electronic Science and Technology of China, in 2009. He is currently with the School of Computer Science, Civil Aviation Flight University of China, Deyang. His research interests include computer software engineering and aviation-oriented computer applications.

...